AlphaBot

Le but du projet est de réaliser un robot capable d'écrire les 26 lettres de l'alphabet à partir d'un codage numérique en base 6, qui pourra être composé grâce à deux switchs. Après validation par un bumper, le robot analysera le code et tracera automatiquement la lettre correspondante.

Scénario :

Le robot reste immobile, attendant la composition d'un code en base 6 à l'aide de deux switches : l'un pour les unités, l'autre pour les dizaines.

Une fois le code saisi, l'utilisateur appuie sur le bumper pour le valider. Les LEDs clignotent pour indiquer la prise en compte du code.

Le robot analyse alors la combinaison obtenue et détermine la lettre correspondante. Si le code est invalide, il retourne à l'étape de composition.

Si le code est valide, le robot avance et effectue les mouvements nécessaires pour tracer la lettre associée.

Une fois le tracé terminé, celui-ci réinitialise la valeur de la variable "code_lettre" afin de permettre à l'utilisateur de réitérer le scénario pour une autre lettre.

Schémas :

Étape 0 :

Les LEDs clignotent brièvement pour signaler le lancement du programme.

Le robot attend l'appuis sur les switch 1 & 2.

‘ L'utilisateur compose le code en **base 6** à l'aide des **deux switchs** (unités et dizaines).’

Étape 1 :

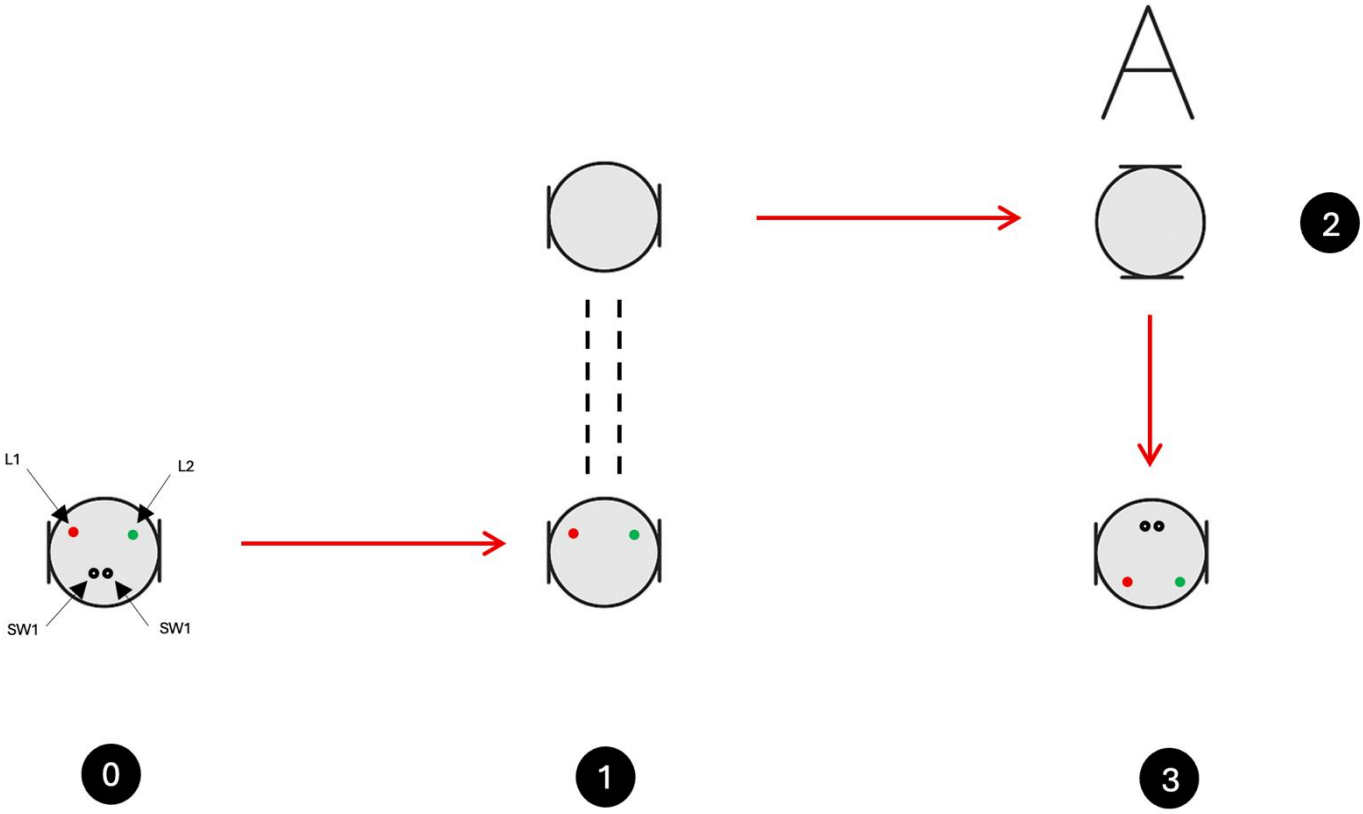
Une fois le code composé et le bumper 1 pressé, les leds clignotent brièvement pour signaler la validation du code, afin que le robot analyse la combinaison et identifie la lettre correspondante.

Étape 2 :

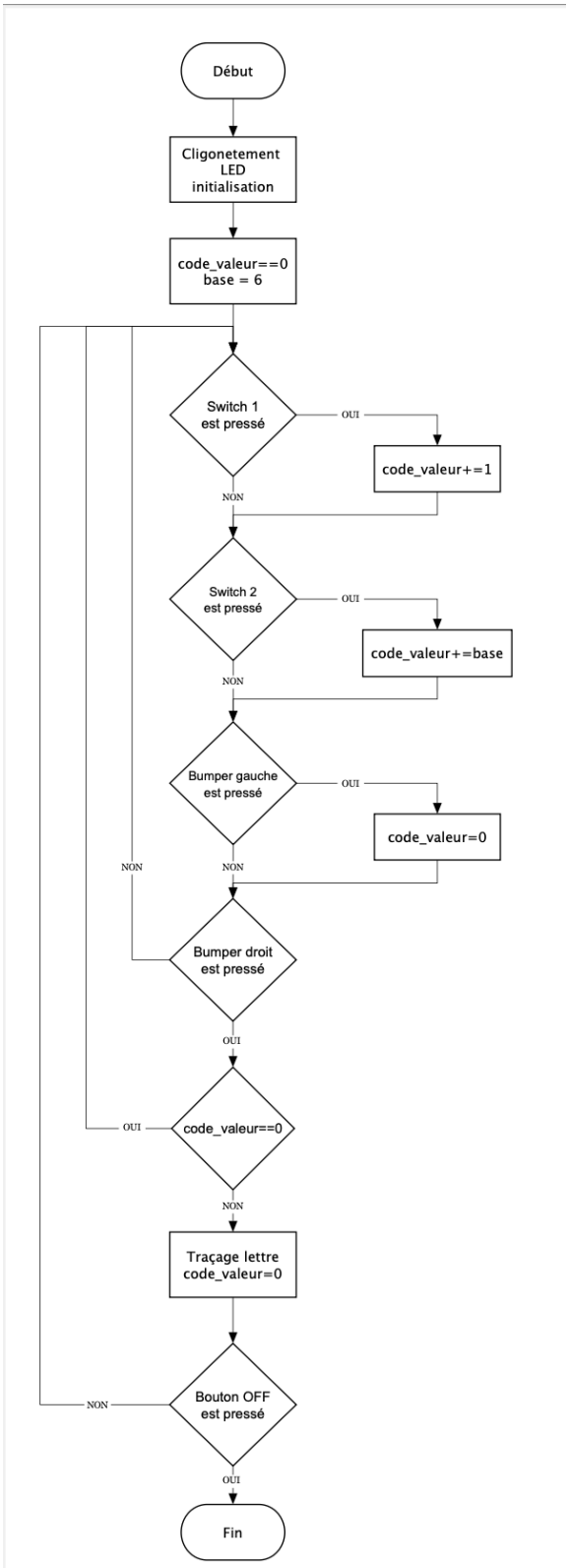
Le robot avance et exécute la fonction d'écriture de la lettre associée.

Étape 3 :

Une fois la lettre tracée, le robot s'arrête et attend pour recommencer un nouveau cycle d'écriture.


A
L1
L2
SW1
SW1
2
0
1
3

Organigramme (Algorithme):



On remarque donc ici que le scénario 1 est définie du départ à la validation du code par le bumper , '**composition et d'analyse du code**' et que le scénario 2, est définie du bloc de validation du code jusqu'au retour à la position initiale. '**exécution et finalisation du tracé de la lettre**'

Choix de la programmation :

GPIO

Les éléments utilisés sont :

- x2 Led
- x2 Switch
- x2 Bumper

La configuration des ports GPIO sera donc :

Led :

```
;Configuration LED 1
ldr r3, = GPIO_PORTF_BASE+GPIO_O_DIR    ; 1 Pin du portF en sortie
;(broche 4 et 5 : 00110000)
ldr r2, = BROCHE4_5
str r2, [r3]
ldr r3, = GPIO_PORTF_BASE+GPIO_O_DEN    ;; Enable Digital Function
ldr r2, = BROCHE4_5
str r2, [r3]

ldr r3, = GPIO_PORTF_BASE+GPIO_O_DR2R   ;; Choix de l'intensité de
;sortie (2mA)
ldr r2, = BROCHE4_5
str r2, [r3]

ldr r3, = adresses_periphs ; on récupère l'adresse du tableau contenant les
    adresses de chaque périphérique
ldr r2, = GPIO_PORTF_BASE + (BROCHE4<<2)  ;; @data Register = @base + (mask<<2)
    ==> LED1
str r2, [r3, #INDICE_LED_1] ; on enregistre l'adresse de la case mémoire gérant
    l'état de la LED 1 dans le tableau adresses_periphs
ldr r2, = GPIO_PORTF_BASE + (BROCHE5<<2)  ;; @data Register = @base + (mask<<2)
    ==> LED2
str r2, [r3, #INDICE_LED_2] ; on enregistre l'adresse de la case mémoire gérant
    l'état de la LED 2 dans le tableau adresses_periphs
;Fin configuration led
```

On va donc ici stocker les adresses des cases mémoires gérant les états des LEDs 1 et 2 dans le tableau ‘adresse_periphs’ aux cases d’indices INDICE_LED_1 et INDICE_LED_2.

Les paramétrages utilisés sont :

```
BROCHE4_5 EQU 0x30, BROCHE5 EQU 0x20 , BROCHE4 EQU 0x10
INDICE_LED_1 EQU 0x0, INDICE_LED_2 EQU 0x4
```

Switch :

```
;Configuration Switches

ldr r3, = GPIO_PORTD_BASE+GPIO_I_PUR    ;; Pul_up
ldr r2, = BROCHE6_7
str r2, [r3]

ldr r3, = GPIO_PORTD_BASE+GPIO_O_DEN    ;; Enable Digital Function
ldr r2, = BROCHE6_7
str r2, [r3]

ldr r3, = adresses_periphs ; on récupère l'adresse du tableau contenant les
    adresses de chaque périphérique
ldr r2, = GPIO_PORTD_BASE + (BROCHE6<<2)  ;; @data Register = @base + (mask<<2)
    ==> Switch 1
str r2, [r3, #INDICE_SWITCH_1] ; on enregistre l'adresse de la case mémoire
    gérant l'état du Switch 1 dans le tableau adresses_periphs

ldr r2, = GPIO_PORTD_BASE + (BROCHE7<<2)  ;; @data Register = @base + (mask<<2)
    ==> Switch 2
str r2, [r3, #INDICE_SWITCH_2] ; on enregistre l'adresse de la case mémoire
    gérant l'état du Switch 2 dans le tableau adresses_periphs

;Fin configuration Switchers
```

On va donc ici stocker les adresses des cases mémoires gérant les états des Switchs 1 et 2 dans le tableau ‘adresse_periphs’ aux cases d’indices INDICE_SWITCH_1 et INDICE_SWITCH_2.

Les paramétrages utilisés sont :

```
BROCHE6_7 EQU 0xC0,BROCHE6   EQU 0x40 , BROCHE7   EQU 0x80

INDICE_SWITCH_1 EQU 0x8 , INDICE_SWITCH_2 EQU 0xC
```

Bumper :

```
;Configuration Bumpers
ldr r3, = GPIO_PORTE_BASE+GPIO_I_PUR    ;; Pul_up
ldr r2, = BROCHE1_0
str r2, [r3]

ldr r3, = GPIO_PORTE_BASE+GPIO_O_DEN    ;; Enable Digital Function
ldr r2, = BROCHE1_0
str r2, [r3]

ldr r3, = adresses_periphs ; on récupére l'adresse du tableau contenant les
    adresses de chaque périphérique
ldr r2, = GPIO_PORTE_BASE + (BROCHE1<<2)  ;; @data Register = @base + (mask<<2)
    ==> Bumper left
str r2, [r3, #INDICE_BUMPER_L] ; on enregistre l'adresse de la case mémoire
    gérant l'état du Bumper left dans le tableau adresses_periphs

ldr r2, = GPIO_PORTE_BASE + (BROCHE0<<2)  ;; @data Register = @base + (mask<<2)
    ==> Bumper right
str r2, [r3, #INDICE_BUMPER_R] ; on enregistre l'adresse de la case mémoire
    gérant l'état du Bumper right dans le tableau adresses_periphs
;Fin configuration Bumpers
```

On va donc ici stocker les adresses des cases mémoires gérant les états des Bumpers 1 et 2 dans le tableau ‘adresse_periphs’ aux cases d’indices INDICE_ BUMPER_1 et INDICE_BUMPER_2.

Les paramétrages utilisés sont :

```
BROCHE1_0 EQU 0x3, BROCHE1 EQU 0x2 , BROCHE0 EQU 0x1
INDICE_BUMPER_1 EQU 0x10, INDICE_BUMPER_2 EQU 0x14
```

Code source

Maxime LE BLAN / Abdelmoumen LAZRAQ

Sommaire

1 main.s **2**

2 config_moteurs.s **11**

3 config_periphs.s **18**

4 fonctions.s **24**

5 deplacements.s **26**

1 main.s

```
1          ;; RK - Evalbot (Cortex M3 de Texas Instrument)
2
3                  AREA    |.text|, CODE, READONLY
4
5                  ENTRY
6                  EXPORT  __main
7
8                  ;; The IMPORT command specifies that a symbol is defined in a
                      shared object at runtime.
9                  IMPORT  MOTEUR_INIT                                     ;
                      initialise les moteurs (configure les pwms + GPIO)
10
11                 IMPORT  MOTEUR_DROIT_ON                                 ; activer le
                      moteur droit
12                 IMPORT  MOTEUR_DROIT_OFF                                ; déactiver le
                      moteur droit
13                 IMPORT  MOTEUR_DROIT_AVANT                              ; moteur droit
                      tourne vers l'avant
14                 IMPORT  MOTEUR_DROIT_ARRIERE            ; moteur droit tourne
                      vers l'arrière
15                 IMPORT  MOTEUR_DROIT_INVERSE            ; inverse le sens de
                      rotation du moteur droit
16
17                 IMPORT  MOTEUR_GAUCHE_ON                                ; activer le
                      moteur gauche
18                 IMPORT  MOTEUR_GAUCHE_OFF                               ; déactiver le
                      moteur gauche
19                 IMPORT  MOTEUR_GAUCHE_AVANT                             ; moteur gauche
                       tourne vers l'avant
20                 IMPORT  MOTEUR_GAUCHE_ARRIERE           ; moteur gauche tourne
                      vers l'arrière
21                 IMPORT  MOTEUR_GAUCHE_INVERSE           ; inverse le sens de
                      rotation du moteur gauche
22
23                 ; Import des fonctions du fichier fonctions.s
24                 IMPORT WAIT_1S
25                 IMPORT WAIT_0_1S
26                 IMPORT INC_CODE_LETTRE
27                 IMPORT INC_CODE_LETTRE_BASE
28                 IMPORT RESET_CODE_LETTRE
29
30                 ; Import des variables du fichier fonctions.s
31                 IMPORT base
32                 IMPORT code_lettre
33
34                 ; Import des fonctions du fichiers config_periphs.s
35                 IMPORT INIT_CLOCK
36                 IMPORT INIT_LEDS
37                 IMPORT INIT_BUMPERS
38                 IMPORT INIT_SWITCHS
39                 IMPORT ALLUMER_LED_1
40                 IMPORT ALLUMER_LED_2
41                 IMPORT ETEINDRE_LED_1
42                 IMPORT ETEINDRE_LED_2
43                 IMPORT SW1_EST_PRESSE
44                 IMPORT SW2_EST_PRESSE
```

```
45                  IMPORT BPR_EST_PRESSE
46                  IMPORT BPL_EST_PRESSE
47
48                  ; Import des variables du fichier config_periphs.s
49                  IMPORT adresses_periphs
50
51                  ; Import des fonctions du fichier deplacements.s
52                  IMPORT DEP_QUART_TOUR_R
53                  IMPORT DEP_QUART_TOUR_L
54                  IMPORT DEP_DEMI_TOUR_R
55                  IMPORT DEP_DEMI_TOUR_L
56                  IMPORT DEP_HUITIEME_TOUR_R
57                  IMPORT DEP_HUITIEME_TOUR_L
58                  IMPORT DEP_SEIZIEME_TOUR_R
59                  IMPORT DEP_SEIZIEME_TOUR_L
60                  IMPORT DEP_LIGNE_DROITE
61                  IMPORT DEP_DEMI_LIGNE_DROITE
62                  IMPORT DEP_DEMI_CERCLE_R
63                  IMPORT DEP_DEMI_CERCLE_L
64                  IMPORT DEP_GRAND_DEMI_CERCLE_R
65                  IMPORT DEP_GRAND_DEMI_CERCLE_L
66
67                  IMPORT DEP_LETTRE_A
68                  IMPORT DEP_LETTRE_B
69                  IMPORT DEP_LETTRE_C
70                  IMPORT DEP_LETTRE_D
71                  IMPORT DEP_LETTRE_E
72                  IMPORT DEP_LETTRE_F
73                  IMPORT DEP_LETTRE_G
74                  IMPORT DEP_LETTRE_H
75                  IMPORT DEP_LETTRE_I
76                  IMPORT DEP_LETTRE_J
77                  IMPORT DEP_LETTRE_K
78                  IMPORT DEP_LETTRE_L
79                  IMPORT DEP_LETTRE_M
80                  IMPORT DEP_LETTRE_N
81                  IMPORT DEP_LETTRE_O
82                  IMPORT DEP_LETTRE_P
83                  IMPORT DEP_LETTRE_Q
84                  IMPORT DEP_LETTRE_R
85                  IMPORT DEP_LETTRE_S
86                  IMPORT DEP_LETTRE_T
87                  IMPORT DEP_LETTRE_U
88                  IMPORT DEP_LETTRE_V
89                  IMPORT DEP_LETTRE_W
90                  IMPORT DEP_LETTRE_X
91                  IMPORT DEP_LETTRE_Y
92                  IMPORT DEP_LETTRE_Z
93
94  __main
95
96                  ; R11 => registre de stockage temporaire des données
97                  ; R12 => registre de stockage temporaire des adresses
98
99
100                 ; R2 => stocke la valeur d'un élément du tableau
                        adresses_periphs
```

```
101                 ; R3 => stocke l'adresse d'un élément du tableau
                        adresses_periphs
102
103                 ; R4 => stocke une valeur temporairement
104
105
106                 ;; BL Branchement vers un lien (sous programme)
107
108                 ; Configuration de la Clock et des périphériques
109                 BL      INIT_CLOCK
110                 BL      INIT_LEDS
111                 BL      INIT_BUMPERS
112                 BL      INIT_SWITCHS
113
114                 ; Configure les PWM + GPIO
115                 BL      MOTEUR_INIT
116
117                 ; ---- DEBUT PROGRAMME ----
118
119                 ; Clignotement par défaut des LEDs pour signifier que le
                        programme est chargé dans l'Evalbot
120                 BL ALLUMER_LED_1
121                 BL ALLUMER_LED_2
122                 BL WAIT_0_1S
123                 BL ETEINDRE_LED_1
124                 BL ETEINDRE_LED_2
125                 BL WAIT_0_1S
126                 BL ALLUMER_LED_1
127                 BL ALLUMER_LED_2
128                 BL WAIT_0_1S
129                 BL ETEINDRE_LED_1
130                 BL ETEINDRE_LED_2
131
132                 ; On initialise code_lettre à 0
133                 BL RESET_CODE_LETTRE
134
135                 ; On initialise R4 à 0 indiquant que l'Evalbot ne tourne pas
                        sur lui même
136
137                 ; ---- DEBUT DE LA BOUCLE PRINCIPALE ----
138
139 MainLoop
140
141                 ; Si le Switch 1 est pressé, on incrémente la variable
                        code_lettre
142                 BL SW1_EST_PRESSE
143                 BNE Si_1
144
145                 BL INC_CODE_LETTRE
146                 ; on fait clignoter la LED 1 pour indiquer un changement de
                        valeur
147                 BL ALLUMER_LED_1
148                 BL WAIT_0_1S
149                 BL WAIT_0_1S
150                 BL WAIT_0_1S
151                 BL ETEINDRE_LED_1
152 Si_1
```

```
153                 ; Si le Switch 2 est pressé, on incrémente la variable
                        code_lettre de la valeur contenu dans "base"
154                 BL SW2_EST_PRESSE
155                 BNE Si_2
156
157                 BL INC_CODE_LETTRE_BASE
158                 ; on fait clignoter la LED 2 pour indiquer un changement de
                        valeur
159                 BL ALLUMER_LED_2
160                 BL WAIT_0_1S
161                 BL WAIT_0_1S
162                 BL WAIT_0_1S
163                 BL ETEINDRE_LED_2
164 Si_2
165
166                 ; Si le Bumper gauche est pressé, on réinitialise code_lettre à
                        0
167                 BL BPL_EST_PRESSE
168                 BNE Si_4
169
170                 BL RESET_CODE_LETTRE
171                 BL ALLUMER_LED_2
172                 BL WAIT_0_1S
173                 BL WAIT_0_1S
174                 BL WAIT_0_1S
175                 BL ETEINDRE_LED_2
176
177 Si_4
178
179                 ; Si le Bumper droit est pressé, alors l'Evalbot va tracer la
                        lettre correspondante à la valeur de code_lettre
180                 BL BPR_EST_PRESSE
181                 BNE.W Si_3
182
183                 ; ---- DEBUT EXECUTION DU TRACAGE DE LA LETTRE CORRESPONDANTE
                        ----
184
185                 ; On lance une animation des LEDs pour indiquer le lancement du
                        processus du traçage de lettre
186                 MOV r11, #0x3
187 While_1
188
189                 BL ALLUMER_LED_1
190                 BL WAIT_0_1S
191                 BL WAIT_0_1S
192                 BL WAIT_0_1S
193                 BL ALLUMER_LED_2
194                 BL WAIT_0_1S
195                 BL WAIT_0_1S
196                 BL WAIT_0_1S
197                 BL ETEINDRE_LED_1
198                 BL WAIT_0_1S
199                 BL WAIT_0_1S
200                 BL WAIT_0_1S
201                 BL ETEINDRE_LED_2
202                 BL WAIT_0_1S
203                 BL WAIT_0_1S
204                 BL WAIT_0_1S
```

```
205                 BL WAIT_0_1S
206                 BL WAIT_0_1S
207
208                 SUBS r11, #1
209                 CMP r11, #0x0
210                 BNE While_1
211
212                 ; On récupère la valeur de code_lettre
213                 LDR r12, = code_lettre
214                 LDRB r11, [r12]
215                 ; On exécute la fonction de déplacement pour la lettre associée
                        à code_lettre
216
217                 ; ----- LIGNE DE TEST TEMPORAIRE -----
218                 ;BL DEP_DEMI_TOUR_R
219                 ; ----- FIN LIGNE DE TEST TEMPORAIRE -----
220
221                 ; Lettre A
222                 CMP r11, #1
223                 BNE Si_5
224
225                 BL DEP_LETTRE_A
226                 ; On réinitialise la variable code_lettre
227                 BL RESET_CODE_LETTRE
228
229 Si_5
230                 ; Lettre B
231                 CMP r11, #2
232                 BNE Si_6
233
234                 BL DEP_LETTRE_B
235                 ; On réinitialise la variable code_lettre
236                 BL RESET_CODE_LETTRE
237
238 Si_6
239                 ; Lettre C
240                 CMP r11, #3
241                 BNE Si_7
242
243                 BL DEP_LETTRE_C
244                 ; On réinitialise la variable code_lettre
245                 BL RESET_CODE_LETTRE
246
247 Si_7
248                 ; Lettre D
249                 CMP r11, #4
250                 BNE Si_8
251
252                 BL DEP_LETTRE_D
253                 ; On réinitialise la variable code_lettre
254                 BL RESET_CODE_LETTRE
255
256 Si_8
257                 ; Lettre E
258                 CMP r11, #5
259                 BNE Si_9
260
261                 BL DEP_LETTRE_E
```

```
262                 ; On réinitialise la variable code_lettre
263                 BL RESET_CODE_LETTRE
264
265 Si_9
266                 ; Lettre F
267                 CMP r11, #6
268                 BNE Si_10
269
270                 BL DEP_LETTRE_F
271                 ; On réinitialise la variable code_lettre
272                 BL RESET_CODE_LETTRE
273
274 Si_10
275                 ; Lettre G
276                 CMP r11, #7
277                 BNE Si_30
278
279                 BL DEP_LETTRE_G
280                 ; On réinitialise la variable code_lettre
281                 BL RESET_CODE_LETTRE
282
283 Si_30
284                 ; Lettre H
285                 CMP r11, #8
286                 BNE Si_11
287
288                 BL DEP_LETTRE_H
289                 ; On réinitialise la variable code_lettre
290                 BL RESET_CODE_LETTRE
291
292 Si_11
293                 ; Lettre I
294                 CMP r11, #9
295                 BNE Si_12
296
297                 BL DEP_LETTRE_I
298                 ; On réinitialise la variable code_lettre
299                 BL RESET_CODE_LETTRE
300
301 Si_12
302                 ; Lettre J
303                 CMP r11, #10
304                 BNE Si_13
305
306                 BL DEP_LETTRE_J
307                 ; On réinitialise la variable code_lettre
308                 BL RESET_CODE_LETTRE
309
310 Si_13
311                 ; Lettre K
312                 CMP r11, #11
313                 BNE Si_14
314
315                 BL DEP_LETTRE_K
316                 ; On réinitialise la variable code_lettre
317                 BL RESET_CODE_LETTRE
318
319 Si_14
```

```
                ; Lettre L
                CMP r11, #12
                BNE Si_15

                BL DEP_LETTRE_L
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_15
                ; Lettre M
                CMP r11, #13
                BNE Si_16

                BL DEP_LETTRE_M
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_16
                ; Lettre N
                CMP r11, #14
                BNE Si_17

                BL DEP_LETTRE_N
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_17
                ; Lettre O
                CMP r11, #15
                BNE Si_18

                BL DEP_LETTRE_O
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_18
                ; Lettre P
                CMP r11, #16
                BNE Si_19

                BL DEP_LETTRE_P
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_19
                ; Lettre Q
                CMP r11, #17
                BNE Si_20

                BL DEP_LETTRE_Q
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_20
                ; Lettre R
                CMP r11, #18
                BNE Si_21

```

```
                BL DEP_LETTRE_R
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_21
                ; Lettre S
                CMP r11, #19
                BNE Si_22

                BL DEP_LETTRE_S
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_22
                ; Lettre T
                CMP r11, #20
                BNE Si_23

                BL DEP_LETTRE_T
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_23
                ; Lettre U
                CMP r11, #21
                BNE Si_24

                BL DEP_LETTRE_U
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_24
                ; Lettre V
                CMP r11, #22
                BNE Si_25

                BL DEP_LETTRE_V
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_25
                ; Lettre W
                CMP r11, #23
                BNE Si_26

                BL DEP_LETTRE_W
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

Si_26
                ; Lettre X
                CMP r11, #24
                BNE Si_27

                BL DEP_LETTRE_X
                ; On réinitialise la variable code_lettre
                BL RESET_CODE_LETTRE

```

```
436 Si_27
437                     ; Lettre Y
438                     CMP r11, #25
439                     BNE Si_28
440 
441                     BL DEP_LETTRE_Y
442                     ; On réinitialise la variable code_lettre
443                     BL RESET_CODE_LETTRE
444 
445 Si_28
446                     ; Lettre Z
447                     CMP r11, #26
448                     BNE Si_29
449 
450                     BL DEP_LETTRE_Z
451                     ; On réinitialise la variable code_lettre
452                     BL RESET_CODE_LETTRE
453 
454 Si_29
455 
456 Si_3
457 
458                     B MainLoop
459 
460                     NOP
461             END
```

2 config_moteurs.s

```
1         ;; RK - Evalbot (Cortex M3 de Texas Instrument);
2 ; programme - Pilotage 2 Moteurs Evalbot par PWM tout en ASM (configure les
     pwms + GPIO)
3
4 ;Les pages se réfèrent au datasheet lm3s9b92.pdf
5
6 ;Cablage :
7 ;pin 10/PD0/PWM0 => input PWM du pont en H DRV8801RT
8 ;pin 11/PD1/PWM1 => input Phase_R  du pont en H DRV8801RT
9 ;pin 12/PD2              => input SlowDecay commune aux 2 ponts en H
10 ;pin 98/PD5              => input Enable 12v du conv DC/DC
11 ;pin 86/PH0/PWM2 => input PWM du 2nd pont en H
12 ;pin 85/PH1/PWM3 => input Phase du 2nd pont en H
13
14 ;; Hexa corresponding values to pin numbers
15 GPIO_0          EQU             0x1
16 GPIO_1          EQU             0x2
17 GPIO_2          EQU             0x4
18 GPIO_5          EQU             0x20
19
20 ;; pour enable clock    0x400FE000
21 SYSCTL_RCGC0    EQU             0x400FE100              ;SYSCTL_RCGC0: offset 0
     x100 (p271 datasheet de lm3s9b92.pdf)
22 SYSCTL_RCGC2    EQU             0x400FE108              ;SYSCTL_RCGC2: offset 0
     x108 (p291 datasheet de lm3s9b92.pdf)
23
24 ;; General-Purpose Input/Outputs (GPIO) configuration
25 PORTD_BASE              EQU             0x40007000
26 GPIODATA_D              EQU             PORTD_BASE
27 GPIODIR_D               EQU             PORTD_BASE+0x00000400
28 GPIODR2R_D              EQU             PORTD_BASE+0x00000500
29 GPIODEN_D               EQU             PORTD_BASE+0x0000051C
30 GPIOPCTL_D              EQU             PORTD_BASE+0x0000052C ; GPIO Port
     Control (GPIOPCTL), offset 0x52C; p444
31 GPIOAFSEL_D             EQU             PORTD_BASE+0x00000420 ; GPIO Alternate
     Function Select (GPIOAFSEL), offset 0x420; p426
32
33 PORTH_BASE              EQU             0x40027000
34 GPIODATA_H              EQU             PORTH_BASE
35 GPIODIR_H               EQU             PORTH_BASE+0x00000400
36 GPIODR2R_H              EQU             PORTH_BASE+0x00000500
37 GPIODEN_H               EQU             PORTH_BASE+0x0000051C
38 GPIOPCTL_H              EQU             PORTH_BASE+0x0000052C ; GPIO Port
     Control (GPIOPCTL), offset 0x52C; p444
39 GPIOAFSEL_H             EQU             PORTH_BASE+0x00000420 ; GPIO Alternate
     Function Select (GPIOAFSEL), offset 0x420; p426
40
41 ;; Pulse Width Modulator (PWM) configuration
42 PWM_BASE                EQU             0x040028000         ;BASE des Block PWM
     p.1138
43 PWMENABLE               EQU             PWM_BASE+0x008      ; p1145
44
45 ;Block PWM0 pour sorties PWM0 et PWM1 (moteur 1)
46 PWM0CTL                 EQU             PWM_BASE+0x040 ;p1167
47 PWM0LOAD                EQU             PWM_BASE+0x050
48 PWM0CMPA                EQU             PWM_BASE+0x058
```

```
PWM0CMPB                EQU             PWM_BASE+0x05C
PWM0GENA                EQU             PWM_BASE+0x060
PWM0GENB                EQU             PWM_BASE+0x064

;Block PWM1 pour sorties PWM1 et PWM2 (moteur 2)
PWM1CTL                 EQU             PWM_BASE+0x080
PWM1LOAD                EQU             PWM_BASE+0x090
PWM1CMPA                EQU             PWM_BASE+0x098
PWM1CMPB                EQU             PWM_BASE+0x09C
PWM1GENA                EQU             PWM_BASE+0x0A0
PWM1GENB                EQU             PWM_BASE+0x0A4


VITESSE                 EQU             0x16D    ; Valeures plus petites => Vitesse plus rapide exemple 0x192 (0x1A2 = 418) (0xD1 = 209) (0x16D = 365)
                                                                                ; Valeures plus grandes => Vitesse moins rapide exemple 0x1B2


                AREA    |.text|, CODE, READONLY
                ENTRY

                ;; The EXPORT command specifies that a symbol can be accessed by other shared objects or executables.
                EXPORT  MOTEUR_INIT
                EXPORT  MOTEUR_DROIT_ON
                EXPORT  MOTEUR_DROIT_OFF
                EXPORT  MOTEUR_DROIT_AVANT
                EXPORT  MOTEUR_DROIT_ARRIERE
                EXPORT  MOTEUR_DROIT_INVERSE
                EXPORT  MOTEUR_GAUCHE_ON
                EXPORT  MOTEUR_GAUCHE_OFF
                EXPORT  MOTEUR_GAUCHE_AVANT
                EXPORT  MOTEUR_GAUCHE_ARRIERE
                EXPORT  MOTEUR_GAUCHE_INVERSE


MOTEUR_INIT
                ldr r6, = SYSCTL_RCGC0
                ldr     r0, [R6]
        ORR     r0, r0, #0x00100000  ;;bit 20 = PWM recoit clock: ON (p271)
        str r0, [r6]

        ;ROM_SysCtlPWMClockSet(SYSCTL_PWMDIV_1);PWM clock is processor clock /1
        ;Je ne fais rien car par defaut = OK!!
        ;*(int *) (0x400FE060)= *(int *)(0x400FE060)...;

        ;RCGC2 :  Enable port D GPIO(p291 ) car Moteur Droit sur port D
                ldr r6, = SYSCTL_RCGC2
                ldr     r0, [R6]
        ORR     r0, r0, #0x08  ;; Enable port D GPIO
        str r0, [r6]

        ;MOT2 : RCGC2 :  Enable port H GPIO  (2eme moteurs)
```

```
100             ldr r6, = SYSCTL_RCGC2
101             ldr     r0, [R6]
102         ORR     r0, r0, #0x80  ;; Enable port H GPIO
103         str r0, [r6]
104
105             nop
106             nop
107             nop
108
109         ;;Pin muxing pour PWM, port D, reg. GPIOPCTL(p444), 4bits de PCM0
                =0001<=>PWM (voir p1261)
110         ;;il faut mettre 1 pour avoir PD0=PWM0 et PD1=PWM1
111             ldr r6, = GPIOPCTL_D
112             ;ldr    r0, [R6]          ;;      *(int *)(0x40007000+0x0000052C)
                    =1;
113         ;ORR    r0, r0, #0x01 ;; Port D, pin 1 = PWM
114             mov     r0, #0x01
115         str r0, [r6]
116
117         ;;MOT2 : Pin muxing pour PWM, port H, reg. GPIOPCTL(p444), 4bits de
                PCM0=0001<=>PWM (voir p1261)
118         ;;il faut mettre mux = 2 pour avoir PH0=PWM2 et PH1=PWM3
119             ldr r6, = GPIOPCTL_H
120             mov     r0, #0x02
121         str r0, [r6]
122
123         ;;Alternate Function Select (p 426), PD0 utilise alernate fonction (PWM
                au dessus)
124         ;;donc PD0 = 1
125             ldr r6, = GPIOAFSEL_D
126             ldr     r0, [R6]          ;*(int *)(0x40007000+0x00000420)= *(
                    int *)(0x40007000+0x00000420) | 0x00000001;
127         ORR     r0, r0, #0x01 ;
128         str r0, [r6]
129
130         ;;MOT2 : Alternate Function Select (p 426), PH0 utilise PWM donc
                Alternate funct
131         ;;donc PH0 = 1
132             ldr r6, = GPIOAFSEL_H
133             ldr     r0, [R6]          ;*(int *)(0x40007000+0x00000420)= *(
                    int *)(0x40007000+0x00000420) | 0x00000001;
134         ORR     r0, r0, #0x01 ;
135         str r0, [r6]
136
137         ;;-----------PWM0 pour moteur 1 connecté à PD0
138         ;;PWM0 produit PWM0 et PWM1 output
139         ;;Config Modes PWM0 + mode GenA + mode GenB
140             ldr r6, = PWM0CTL
141             mov     r0, #2          ;Mode up-down-up-down, pas synchro
142         str r0, [r6]
143
144             ldr r6, =PWM0GENA ;en decomptage, qd comparateurA = compteur =>
                    sortie pwmA=0
145                                       ;en comptage croissant, qd
                                              comparateurA = compteur =>
                                              sortie pwmA=1
146             mov     r0,     #0x0B0  ;0B0=10110000 => ACTCMPBD=00 (B down:
                    rien), ACTCMPBU=00(B up rien)
```

```
147                 str r0, [r6]     ;ACTCMPAD=10 (A down:pwmA low), ACTCMPAU=11 (A
                        up:pwmA high) , ACTLOAD=00,ACTZERO=00
148
149                 ldr r6, =PWM0GENB;en comptage croissant, qd comparateurB =
                        compteur => sortie pwmA=1
150                 mov      r0,      #0x0B00 ;en decomptage, qd comparateurB =
                        compteur => sortie pwmB=0
151                 str r0, [r6]
152             ;Config Compteur, comparateur A et comparateur B
153             ;;#define PWM_PERIOD (ROM_SysCtlClockGet() / 16000),
154             ;;en mesure : SysCtlClockGet=0F42400h, /16=0x3E8,
155             ;;on divise par 2 car moteur 6v sur alim 12v
156                 ldr      r6, =PWM0LOAD ;PWM0LOAD=periode/2 =0x1F4
157                 mov r0, #0x1F4
158                 str      r0,[r6]
159
160                 ldr      r6, =PWM0CMPA ;Valeur rapport cyclique : pour 10% => 1
                        C2h si clock = 0F42400
161                 mov      r0, #VITESSE
162                 str      r0, [r6]
163
164                 ldr      r6, =PWM0CMPB ;PWM0CMPB recoit meme valeur. (rapport
                        cyclique depend de CMPA)
165                 mov      r0,      #0x1F4
166                 str      r0,      [r6]
167
168             ;Control PWM : active PWM Generator 0 (p1167): Enable+up/down + Enable
                    counter debug mod
169                 ldr      r6, =PWM0CTL
170                 ldr      r0, [r6]
171                 ORR      r0,      r0,      #0x07
172                 str      r0,      [r6]
173
174             ;;-----------PWM2 pour moteur 2 connecté à PH0
175             ;;PWM1block produit PWM2 et PWM3 output
176                 ;;Config Modes PWM2 + mode GenA + mode GenB
177                 ldr r6, = PWM1CTL
178                 mov      r0, #2           ;Mode up-down-up-down, pas synchro
179             str r0, [r6]     ;*(int *)(0x40028000+0x040)=2;
180
181                 ldr r6, =PWM1GENA ;en decomptage, qd comparateurA = compteur =>
                         sortie pwmA=0
182                                             ;en comptage croissant, qd
                                                    comparateurA = compteur =>
                                                    sortie pwmA=1
183                 mov      r0,      #0x0B0   ;0B0=10110000 => ACTCMPBD=00 (B down:
                        rien), ACTCMPBU=00(B up rien)
184                 str r0, [r6]     ;ACTCMPAD=10 (A down:pwmA low), ACTCMPAU=11 (A
                        up:pwmA high) , ACTLOAD=00,ACTZERO=00
185
186                 ;*(int *)(0x40028000+0x060)=0x0B0; //
187                 ldr r6, =PWM1GENB        ;*(int *)(0x40028000+0x064)=0x0B00;
188                 mov      r0,      #0x0B00 ;en decomptage, qd comparateurB =
                        compteur => sortie pwmB=0
189                 str r0, [r6]     ;en comptage croissant, qd comparateurB =
                        compteur => sortie pwmA=1
190             ;Config Compteur, comparateur A et comparateur B
191             ;;#define PWM_PERIOD (ROM_SysCtlClockGet() / 16000),
```

```
192         ;;en mesure : SysCtlClockGet=0F42400h, /16=0x3E8,
193         ;;on divise par 2 car moteur 6v sur alim 12v
194                 ;*(int *)(0x40028000+0x050)=0x1F4; //PWM0LOAD=periode/2 =0x1F4
195                 ldr     r6, =PWM1LOAD
196                 mov r0, #0x1F4
197                 str     r0,[r6]
198
199                 ldr     r6, =PWM1CMPA ;Valeur rapport cyclique : pour 10% => 1
                        C2h si clock = 0F42400
200                 mov     r0,     #VITESSE
201                 str     r0, [r6]  ;*(int *)(0x40028000+0x058)=0x01C2;
202
203                 ldr     r6, =PWM1CMPB ;PWM0CMPB recoit meme valeur. (CMPA
                        depend du rapport cyclique)
204                 mov     r0,     #0x1F4  ; *(int *)(0x40028000+0x05C)=0x1F4;
205                 str     r0,     [r6]
206
207         ;Control PWM : active PWM Generator 0 (p1167): Enable+up/down + Enable
                counter debug mod
208                 ldr     r6, =PWM1CTL
209                 ldr     r0, [r6]        ;*(int *) (0x40028000+0x40)= *(int *)(0
                        x40028000+0x40) | 0x07;
210                 ORR     r0,     r0,     #0x07
211                 str     r0,     [r6]
212
213         ;;-----Fin config des PWMs
214
215         ;PORT D OUTPUT pin0 (pwm)=pin1(direction)=pin2(slow decay)=pin5(12v
                enable)
216                 ldr     r6, =GPIODIR_D
217                 ldr     r0, [r6]
218                 ORR     r0,     #(GPIO_0+GPIO_1+GPIO_2+GPIO_5)
219                 str     r0,[r6]
220         ;Port D, 2mA les meme
221                 ldr     r6, =GPIODR2R_D ;
222                 ldr     r0, [r6]
223                 ORR     r0,     #(GPIO_0+GPIO_1+GPIO_2+GPIO_5)
224                 str     r0,[r6]
225         ;Port D, Digital Enable
226                 ldr     r6, =GPIODEN_D ;
227                 ldr     r0, [r6]
228                 ORR     r0,     #(GPIO_0+GPIO_1+GPIO_2+GPIO_5)
229                 str     r0,[r6]
230         ;Port D : mise à 1 de slow Decay et 12V et mise à 0 pour dir et pwm
231                 ldr     r6, =(GPIODATA_D+((GPIO_0+GPIO_1+GPIO_2+GPIO_5)<<2))
232                 mov     r0, #(GPIO_2+GPIO_5) ; #0x24
233                 str     r0,[r6]
234
235         ;MOT2, PH1 pour sens moteur ouput
236                 ldr     r6, =GPIODIR_H
237                 mov     r0,     #0x03   ;
238                 str     r0,[r6]
239         ;Port H, 2mA les meme
240                 ldr     r6, =GPIODR2R_H
241                 mov r0, #0x03
242                 str     r0,[r6]
243         ;Port H, Digital Enable
244                 ldr     r6, =GPIODEN_H
```

```
245                 mov r0, #0x03
246                 str     r0,[r6]
247         ;Port H : mise à 1 pour dir
248                 ldr     r6, =(GPIODATA_H +(GPIO_1<<2))
249                 mov     r0, #0x02
250                 str     r0,[r6]
251
252                 BX      LR      ; FIN du sous programme d'init.
253
254 ;Enable PWM0 (bit 0) et PWM2 (bit 2) p1145
255 ;Attention ici c'est les sorties PWM0 et PWM2
256 ;qu'on controle, pas les blocks PWM0 et PWM1!!!
257 MOTEUR_DROIT_ON
258                 ;Enable sortie PWM0 (bit 0), p1145
259                 ldr     r6,     =PWMENABLE
260                 ldr r0, [r6]
261                 orr r0, #0x01 ;bit 0 à 1
262                 str     r0,     [r6]
263                 BX      LR
264
265 MOTEUR_DROIT_OFF
266                 ldr     r6,     =PWMENABLE
267                 ldr r0, [r6]
268                 and     r0,     #0x0E   ;bit 0 à 0
269                 str     r0,     [r6]
270                 BX      LR
271
272 MOTEUR_GAUCHE_ON
273                 ldr     r6,     =PWMENABLE
274                 ldr     r0, [r6]
275                 orr     r0,     #0x04   ;bit 2 à 1
276                 str     r0,     [r6]
277                 BX      LR
278
279 MOTEUR_GAUCHE_OFF
280                 ldr     r6,     =PWMENABLE
281                 ldr     r0,     [r6]
282                 and     r0,     #0x0B   ;bit 2 à 0
283                 str     r0,     [r6]
284                 BX      LR
285
286 MOTEUR_DROIT_ARRIERE
287                 ;Inverse Direction (GPIO_D1)
288                 ldr     r6, =(GPIODATA_D+(GPIO_1<<2))
289                 mov     r0, #0
290                 str     r0,[r6]
291                 BX      LR
292
293 MOTEUR_DROIT_AVANT
294                 ;Inverse Direction (GPIO_D1)
295                 ldr     r6, =(GPIODATA_D+(GPIO_1<<2))
296                 mov     r0, #2
297                 str     r0,[r6]
298                 BX      LR
299
300 MOTEUR_GAUCHE_ARRIERE
301                 ;Inverse Direction (GPIO_D1)
302                 ldr     r6, =(GPIODATA_H+(GPIO_1<<2))
```

```
303                 mov     r0, #2 ; contraire du moteur Droit
304                 str     r0,[r6]
305                 BX      LR
306
307 MOTEUR_GAUCHE_AVANT
308                 ;Inverse Direction (GPIO_D1)
309                 ldr     r6, =(GPIODATA_H+(GPIO_1<<2))
310                 mov     r0, #0
311                 str     r0,[r6]
312                 BX      LR
313
314 MOTEUR_DROIT_INVERSE
315                 ;Inverse Direction (GPIO_D1)
316                 ldr     r6, =(GPIODATA_D+(GPIO_1<<2))
317                 ldr     r1, [r6]
318                 EOR     r0, r1, #GPIO_1
319                 str     r0,[r6]
320                 BX      LR
321
322 MOTEUR_GAUCHE_INVERSE
323                 ;Inverse Direction (GPIO_D1)
324                 ldr     r6, =(GPIODATA_H+(GPIO_1<<2))
325                 ldr     r1, [r6]
326                 EOR     r0, r1, #GPIO_1
327                 str     r0,[r6]
328                 BX      LR
329
330                 END
```

3 config_periphs.s

```
1         ;; RK - Evalbot (Cortex M3 de Texas Instrument)
2
3                 AREA    |.text|, CODE, READONLY
4
5 ; This register controls the clock gating logic in normal Run mode
6 SYSCTL_PERIPH_GPIO EQU          0x400FE108      ; SYSCTL_RCGC2_R (p291
      datasheet de lm3s9b92.pdf)
7
8 ; The GPIODATA register is the data register
9 GPIO_PORTF_BASE         EQU             0x40025000      ; GPIO Port F (APB)
      base: 0x4002.5000 (p416 datasheet de lm3s9B92.pdf)
10
11 ; The GPIODATA register is the data register
12 GPIO_PORTD_BASE         EQU             0x40007000              ; GPIO Port D (
      APB) base: 0x4000.7000 (p416 datasheet de lm3s9B92.pdf)
13
14 GPIO_PORTE_BASE         EQU             0x40024000              ; GPIO Port E (
      APB) base: 0x40024000 (p416 datasheet de lm3s9B92.pdf)
15
16 ; configure the corresponding pin to be an output
17 ; all GPIO pins are inputs by default
18 GPIO_O_DIR              EQU     0x00000400  ; GPIO Direction (p417 datasheet de
       lm3s9B92.pdf)
19
20 ; The GPIODR2R register is the 2-mA drive control register
21 ; By default, all GPIO pins have 2-mA drive.
22 GPIO_O_DR2R             EQU     0x00000500  ; GPIO 2-mA Drive Select (p428
      datasheet de lm3s9B92.pdf)
23
24 ; Digital enable register
25 ; To use the pin as a digital input or output, the corresponding GPIODEN bit
      must be set.
26 GPIO_O_DEN              EQU     0x0000051C  ; GPIO Digital Enable (p437
      datasheet de lm3s9B92.pdf)
27
28 ; Pul_up
29 GPIO_I_PUR              EQU     0x00000510  ; GPIO Pull-Up (p432 datasheet de
      lm3s9B92.pdf)
30
31 ; Broches select
32 BROCHE4_5                       EQU             0x30            ; led1 & led2
      sur broche 4 et 5
33 BROCHE5                         EQU     0x20            ; led 2
34 BROCHE4                         EQU     0x10            ; led 1
35 BROCHE6                         EQU     0x40            ; switch 1 (0x40)
36 BROCHE7                         EQU     0x80            ; switch 2 (0x40)
37 BROCHE6_7                       EQU             0xC0            ; both switches
38 BROCHE0                         EQU             0x1                     ;
      bumper right
39 BROCHE1                         EQU             0x2                     ;
      bumper left
40 BROCHE1_0                       EQU             0x3                     ; both
      bumpers
41
42 ; éteindre une LED
43 ETEINDRE                        EQU     0x0
```

```
44
45  ; Indice de chaque capteur pour les tableaux adresses_periphs et
        valeurs_periphs
46  INDICE_LED_1            EQU             0x0
47  INDICE_LED_2            EQU             0x4
48  INDICE_BUMPER_R         EQU             0x8
49  INDICE_BUMPER_L         EQU             0xC
50  INDICE_SWITCH_1         EQU             0x10
51  INDICE_SWITCH_2         EQU             0x14
52
53
54
55
56                  ENTRY
57
58                  EXPORT INIT_CLOCK
59                  EXPORT INIT_LEDS
60                  EXPORT INIT_BUMPERS
61                  EXPORT INIT_SWITCHS
62                  EXPORT ALLUMER_LED_1
63                  EXPORT ALLUMER_LED_2
64                  EXPORT ETEINDRE_LED_1
65                  EXPORT ETEINDRE_LED_2
66                  EXPORT SW1_EST_PRESSE
67                  EXPORT SW2_EST_PRESSE
68                  EXPORT BPR_EST_PRESSE
69                  EXPORT BPL_EST_PRESSE
70
71                  EXPORT adresses_periphs
72
73
74  ; CONFIGURATION CLOCK
75  INIT_CLOCK
76
77                  ; ;; Enable the Port F & D peripheral clock              (p291
                        datasheet de lm3s9B96.pdf)
78                  ; ;;
79                  ldr r3, = SYSCTL_PERIPH_GPIO                    ;; RCGC2
80          mov r2, #0x00000038                                     ;; Enable clock
              sur GPIO D, E et F où sont branchés les leds (0x38 == 0b111000)
81                  ; ;;


                                                (GPIO::FEDCBA)
82          str r2, [r3]
83
84                  ; ;; "There must be a delay of 3 system clocks before any GPIO
                        reg. access  (p413 datasheet de lm3s9B92.pdf)
85                  nop
                                                ;; tres tres important....
86                  nop
87                  nop
                                                ;; pas necessaire en simu ou en
                        debbug step by step...
88
89                  BX LR
90
```

```
91  ; CONFIGURATION LED 1
92  INIT_LEDS
93
94          ldr r3, = GPIO_PORTF_BASE+GPIO_O_DIR    ;; 1 Pin du portF en sortie (
                broche 4 et 5 : 00110000)
95          ldr r2, = BROCHE4_5
96          str r2, [r3]
97
98                  ldr r3, = GPIO_PORTF_BASE+GPIO_O_DEN    ;; Enable Digital
                        Function
99          ldr r2, = BROCHE4_5
100         str r2, [r3]
101
102                 ldr r3, = GPIO_PORTF_BASE+GPIO_O_DR2R   ;; Choix de l'intensité
                         de sortie (2mA)
103         ldr r2, = BROCHE4_5
104         str r2, [r3]
105
106                 LDR r3, = adresses_periphs ; on récupère l'adresse du tableau
                        contenant les adresses de chaque périphérique
107                 LDR r2, = GPIO_PORTF_BASE + (BROCHE4<<2)  ;; @data Register =
                        @base + (mask<<2) ==> LED1
108                 STR r2, [r3, #INDICE_LED_1] ; on enregistre l'adresse de la
                        case mémoire gérant l'état de la LED 1 dans le tableau
                        adresses_periphs
109
110                 LDR r2, = GPIO_PORTF_BASE + (BROCHE5<<2)  ;; @data Register =
                        @base + (mask<<2) ==> LED2
111                 STR r2, [r3, #INDICE_LED_2] ; on enregistre l'adresse de la
                        case mémoire gérant l'état de la LED 2 dans le tableau
                        adresses_periphs
112
113                 BX LR
114
115 INIT_BUMPERS
116
117                 ldr r3, = GPIO_PORTE_BASE+GPIO_I_PUR    ;; Pul_up
118         ldr r2, = BROCHE1_0
119         str r2, [r3]
120
121                 ldr r3, = GPIO_PORTE_BASE+GPIO_O_DEN    ;; Enable Digital
                        Function
122         ldr r2, = BROCHE1_0
123         str r2, [r3]
124
125                 LDR r3, = adresses_periphs ; on récupère l'adresse du tableau
                        contenant les adresses de chaque périphérique
126                 LDR r2, = GPIO_PORTE_BASE + (BROCHE1<<2)  ;; @data Register =
                        @base + (mask<<2) ==> Bumper left
127                 STR r2, [r3, #INDICE_BUMPER_L] ; on enregistre l'adresse de la
                        case mémoire gérant l'état du Bumper left dans le tableau
                        adresses_periphs
128
129                 LDR r2, = GPIO_PORTE_BASE + (BROCHE0<<2)  ;; @data Register =
                        @base + (mask<<2) ==> Bumper right
130                 STR r2, [r3, #INDICE_BUMPER_R] ; on enregistre l'adresse de la
                        case mémoire gérant l'état du Bumper right dans le tableau
                        adresses_periphs
```

```
131
132                 BX LR
133
134 INIT_SWITCHS
135
136                 ldr r3, = GPIO_PORTD_BASE+GPIO_I_PUR    ;; Pul_up
137         ldr r2, = BROCHE6_7
138         str r2, [r3]
139
140                 ldr r3, = GPIO_PORTD_BASE+GPIO_O_DEN    ;; Enable Digital
                        Function
141         ldr r2, = BROCHE6_7
142         str r2, [r3]
143
144                 LDR r3, = adresses_periphs ; on récupère l'adresse du tableau
                        contenant les adresses de chaque périphérique
145                 LDR r2, = GPIO_PORTD_BASE + (BROCHE6<<2)  ;; @data Register =
                        @base + (mask<<2) ==> Switch 1
146                 STR r2, [r3, #INDICE_SWITCH_1] ; on enregistre l'adresse de la
                        case mémoire gérant l'état du Switch 1 dans le tableau
                        adresses_periphs
147
148                 LDR r2, = GPIO_PORTD_BASE + (BROCHE7<<2)  ;; @data Register =
                        @base + (mask<<2) ==> Switch 2
149                 STR r2, [r3, #INDICE_SWITCH_2] ; on enregistre l'adresse de la
                        case mémoire gérant l'état du Switch 2 dans le tableau
                        adresses_periphs
150
151                 BX LR
152
153 ALLUMER_LED_1
154
155                 LDR r3, = adresses_periphs
156                 LDR r3, [r3, #INDICE_LED_1] ; on enregistre l'adresse de la
                        case mémoire gérant l'état de la LED 1 dans R3
157                 MOV r2, #BROCHE4
158
159                 STR r2, [r3] ; on allume la LED 1
160
161                 BX LR
162
163 ALLUMER_LED_2
164
165                 LDR r3, = adresses_periphs
166                 LDR r3, [r3, #INDICE_LED_2] ; on enregistre l'adresse de la
                        case mémoire gérant l'état de la LED 2 dans R3
167                 MOV r2, #BROCHE5
168
169                 STR r2, [r3] ; on allume la LED 2
170
171                 BX LR
172
173 ETEINDRE_LED_1
174
175                 LDR r3, = adresses_periphs
176                 LDR r3, [r3, #INDICE_LED_1] ; on enregistre l'adresse de la
                        case mémoire gérant l'état de la LED 1 dans R3
177                 MOV r2, #0x0
```

```
178
179                 STR r2, [r3] ; on éteint la LED 1
180
181                 BX LR
182
183 ETEINDRE_LED_2
184
185                 LDR r3, = adresses_periphs
186                 LDR r3, [r3, #INDICE_LED_2] ; on enregistre l'adresse de la
                        case mémoire gérant l'état de la LED 2 dans R3
187                 MOV r2, #0x0
188
189                 STR r2, [r3] ; on allume la LED 2
190
191                 BX LR
192
193 SW1_EST_PRESSE
194
195                 LDR r3, = adresses_periphs
196                 LDR r3, [r3, #INDICE_SWITCH_1] ; on enregistre l'adresse de la
                        case mémoire gérant l'état du Switch 1 dans R3
197                 LDR r2, [r3] ; on récupère la valeur d'état du Switch 1
198
199                 CMP r2, #0x0 ; on vérifie si le Switch 1 est actuellement
                        pressé (s'il est pressé, sa valeur d'état est 0, 1 sinon)
200
201                 BX LR
202
203 SW2_EST_PRESSE
204
205                 LDR r3, = adresses_periphs
206                 LDR r3, [r3, #INDICE_SWITCH_2] ; on enregistre l'adresse de la
                        case mémoire gérant l'état du Switch 2 dans R3
207                 LDR r2, [r3] ; on récupère la valeur d'état du Switch 2
208
209                 CMP r2, #0x0 ; on vérifie si le Switch 2 est actuellement
                        pressé (s'il est pressé, sa valeur d'état est 0, 1 sinon)
210
211                 BX LR
212
213 BPR_EST_PRESSE
214
215                 LDR r3, = adresses_periphs
216                 LDR r3, [r3, #INDICE_BUMPER_R] ; on enregistre l'adresse de la
                        case mémoire gérant l'état du Bumper droit dans R3
217                 LDR r2, [r3] ; on récupère la valeur d'état du Bumper droit
218
219                 CMP r2, #0x0 ; on vérifie si le Bumper droit est actuellement
                        pressé (s'il est pressé, sa valeur d'état est 0, 1 sinon)
220
221                 BX LR
222
223 BPL_EST_PRESSE
224
225                 LDR r3, = adresses_periphs
226                 LDR r3, [r3, #INDICE_BUMPER_L] ; on enregistre l'adresse de la
                        case mémoire gérant l'état du Bumper gauche dans R3
227                 LDR r2, [r3] ; on récupère la valeur d'état du Bumper gauche
```

```
228
229                 CMP r2, #0x0 ; on vérifie si le Bumper gauche est actuellement
                        pressé (s'il est pressé, sa valeur d'état est 0, 1 sinon)
230
231                 BX LR
232
233                 AREA variables, DATA, READWRITE
234 adresses_periphs SPACE 24
235
236                 END
```

4 fonctions.s

```
1           ;; RK - Evalbot (Cortex M3 de Texas Instrument)
2                   AREA    |.text|, CODE, READONLY
3
4   ; blinking frequency
5   FREQUENCE_7             EQU     0x00989680 ; FREQUENCE = 10e7 Hz
6   FREQUENCE_65            EQU     0x004C4B40 ; FREQUENCE = 5 * 10e6 Hz (environ 1
        s)
7   FREQUENCE_6             EQU     0x000F4240 ; FREQUENCE = 10e6 Hz
8   FREQUENCE_55            EQU     0x0007A120 ; FREQUENCE = 5 * 10e5 Hz
9   FREQUENCE_5             EQU     0x000186A0 ; FREQUENCE = 10e5 Hz
10  FREQUENCE_66666         EQU     0x0001046A ; FREQUENCE = 66 666 Hz
11
12  ; R2 => registre de stockage temporaire des données
13  ; R3 => registre de stockage temporaire des adresses
14
15                  ENTRY
16                  ; export des fonctions
17                  EXPORT WAIT_1S
18                  EXPORT WAIT_0_1S
19                  EXPORT INC_CODE_LETTRE
20                  EXPORT INC_CODE_LETTRE_BASE
21                  EXPORT RESET_CODE_LETTRE
22
23
24                  ; export des variables
25                  EXPORT code_lettre
26                  EXPORT base
27
28
29                  ; Fonction : Attendre 1 seconde
30  WAIT_1S
31                  ldr r2, =FREQUENCE_65
32  wait_1s
33                  subs r2, #1
34          bne wait_1s
35                  BX LR
36
37                  ; Fonction : Attendre 0.1 seconde
38  WAIT_0_1S
39                  ldr r2, =FREQUENCE_55
40  wait_0_1s
41                  subs r2, #1
42          bne wait_0_1s
43                  BX LR
44
45  RESET_CODE_LETTRE
46
47                  LDR r3, =code_lettre ; on récupère l'adresse de code_lettre
                        dans r3
48                  MOV r2, #0x0
49                  STRB r2, [r3] ; on initialise code_lettre à 0
50
51                  BX LR
52
53  INC_CODE_LETTRE
```

```
54                  ldr r3, =code_lettre ; on récupère l'adresse de code_lettre
                        dans r3
55                  LDRB r2, [r3] ; on récupère la valeur de code_lettre dans r2
56                  ADD r2, r2, #1 ; code_lettre = code_lettre + 1
57                  STRB r2, [r3] ; on enregistre la nouvelle valeur de code_lettre
                        dans sa variable dans la mémoire
58                  BX LR
59
60  INC_CODE_LETTRE_BASE
61                  LDR r3, =code_lettre ; on récupère l'adresse de code_lettre
                        dans r3
62                  LDRB r2, [r3] ; on récupère la valeur de code_lettre dans r2
63                  LDR r5, =base ; adresse de base
64                  LDRB r4, [r5] ; r4 contient base
65
66                  ADD r2, r2, r4 ; code_lettre = code_lettre + 5
67                  STRB r2, [r3]
68                  BX LR
69
70                  AREA constantes, DATA, READONLY
71  base    DCB             6
72
73                  AREA variables, DATA, READWRITE
74  code_lettre SPACE 1
75
76          END
```

5 deplacements.s

```
1                   AREA    |.text|, CODE, READONLY
2
3                   ENTRY
4
5                   ; Export des déplacements de base
6                   EXPORT DEP_QUART_TOUR_R
7                   EXPORT DEP_QUART_TOUR_L
8                   EXPORT DEP_DEMI_TOUR_R
9                   EXPORT DEP_DEMI_TOUR_L
10                  EXPORT DEP_HUITIEME_TOUR_R
11                  EXPORT DEP_HUITIEME_TOUR_L
12                  EXPORT DEP_SEIZIEME_TOUR_R
13                  EXPORT DEP_SEIZIEME_TOUR_L
14                  EXPORT DEP_LIGNE_DROITE
15                  EXPORT DEP_DEMI_LIGNE_DROITE
16                  EXPORT DEP_DEMI_CERCLE_R
17                  EXPORT DEP_DEMI_CERCLE_L
18                  EXPORT DEP_GRAND_DEMI_CERCLE_R
19                  EXPORT DEP_GRAND_DEMI_CERCLE_L
20
21                  ; Export des déplacements traçants les lettres
22                  EXPORT DEP_LETTRE_A
23                  EXPORT DEP_LETTRE_B
24                  EXPORT DEP_LETTRE_C
25                  EXPORT DEP_LETTRE_D
26                  EXPORT DEP_LETTRE_E
27                  EXPORT DEP_LETTRE_F
28                  EXPORT DEP_LETTRE_G
29                  EXPORT DEP_LETTRE_H
30                  EXPORT DEP_LETTRE_I
31                  EXPORT DEP_LETTRE_J
32                  EXPORT DEP_LETTRE_K
33                  EXPORT DEP_LETTRE_L
34                  EXPORT DEP_LETTRE_M
35                  EXPORT DEP_LETTRE_N
36                  EXPORT DEP_LETTRE_O
37                  EXPORT DEP_LETTRE_P
38                  EXPORT DEP_LETTRE_Q
39                  EXPORT DEP_LETTRE_R
40                  EXPORT DEP_LETTRE_S
41                  EXPORT DEP_LETTRE_T
42                  EXPORT DEP_LETTRE_U
43                  EXPORT DEP_LETTRE_V
44                  EXPORT DEP_LETTRE_W
45                  EXPORT DEP_LETTRE_X
46                  EXPORT DEP_LETTRE_Y
47                  EXPORT DEP_LETTRE_Z
48
49                  ;; The IMPORT command specifies that a symbol is defined in a
                        shared object at runtime.
50                  IMPORT  MOTEUR_INIT                                     ;
                        initialise les moteurs (configure les pwms + GPIO)
51
52                  IMPORT  MOTEUR_DROIT_ON                          ; activer le
                        moteur droit
```

```
53                  IMPORT  MOTEUR_DROIT_OFF                        ; déactiver le
                        moteur droit
54                  IMPORT  MOTEUR_DROIT_AVANT                      ; moteur droit
                        tourne vers l'avant
55                  IMPORT  MOTEUR_DROIT_ARRIERE            ; moteur droit tourne
                        vers l'arrière
56                  IMPORT  MOTEUR_DROIT_INVERSE            ; inverse le sens de
                        rotation du moteur droit
57
58                  IMPORT  MOTEUR_GAUCHE_ON                        ; activer le
                        moteur gauche
59                  IMPORT  MOTEUR_GAUCHE_OFF                       ; déactiver le
                        moteur gauche
60                  IMPORT  MOTEUR_GAUCHE_AVANT                     ; moteur gauche
                         tourne vers l'avant
61                  IMPORT  MOTEUR_GAUCHE_ARRIERE           ; moteur gauche tourne
                        vers l'arrière
62                  IMPORT  MOTEUR_GAUCHE_INVERSE           ; inverse le sens de
                        rotation du moteur gauche
63
64                  ; Import des fonctions du fichier fonctions.s
65                  IMPORT WAIT_1S
66                  IMPORT WAIT_0_1S
67
68                  ; R4 => stocke une valeur temporairement
69
70  DEP_QUART_TOUR_R
71
72                  PUSH {LR} ; on enregistre l'adresse de l'instruction suivante
                        du programme main.s pour pouvoir y revenir à la fin
73                                  ; de l'exécution de la fonction
                                        DEP_QUART_TOUR_R
74
75                  BL MOTEUR_DROIT_ON
76                  BL MOTEUR_GAUCHE_ON
77                  BL MOTEUR_DROIT_ARRIERE
78                  BL MOTEUR_GAUCHE_AVANT
79
80                  BL WAIT_1S
81                  BL WAIT_1S
82                  BL WAIT_0_1S
83                  BL WAIT_0_1S
84                  BL WAIT_0_1S
85                  BL WAIT_0_1S
86                  BL MOTEUR_DROIT_OFF
87                  BL MOTEUR_GAUCHE_OFF
88
89                  POP {LR} ; on récupère l'adresse de l'instruction suivante du
                        programme main.s pour pouvoir y revenir
90
91                  BX LR
92
93  DEP_QUART_TOUR_L
94
95                  PUSH {LR} ; on enregistre l'adresse de l'instruction suivante
                        du programme main.s pour pouvoir y revenir à la fin
96                                  ; de l'exécution de la fonction
                                        DEP_QUART_TOUR_L
```

```
97
98                  BL MOTEUR_DROIT_ON
99                  BL MOTEUR_GAUCHE_ON
100                 BL MOTEUR_DROIT_AVANT
101                 BL MOTEUR_GAUCHE_ARRIERE
102
103                 BL WAIT_1S
104                 BL WAIT_1S
105                 BL WAIT_0_1S
106                 BL WAIT_0_1S
107                 BL WAIT_0_1S
108                 BL WAIT_0_1S
109                 BL MOTEUR_DROIT_OFF
110                 BL MOTEUR_GAUCHE_OFF
111
112                 POP {LR} ; on récupère l'adresse de l'instruction suivante du
                        programme main.s pour pouvoir y revenir
113
114                 BX LR
115
116 DEP_DEMI_TOUR_R
117
118                 PUSH {LR}
119
120                 BL MOTEUR_DROIT_ON
121                 BL MOTEUR_GAUCHE_ON
122                 BL MOTEUR_DROIT_ARRIERE
123                 BL MOTEUR_GAUCHE_AVANT
124
125                 BL WAIT_1S
126                 BL WAIT_1S
127                 BL WAIT_1S
128                 BL WAIT_1S
129                 BL WAIT_0_1S
130                 BL WAIT_0_1S
131                 BL WAIT_0_1S
132                 BL WAIT_0_1S
133                 BL WAIT_0_1S
134                 BL WAIT_0_1S
135                 BL WAIT_0_1S
136                 BL WAIT_0_1S
137                 BL MOTEUR_DROIT_OFF
138                 BL MOTEUR_GAUCHE_OFF
139
140                 POP {LR}
141
142                 BX LR
143
144 DEP_DEMI_TOUR_L
145
146                 PUSH {LR}
147
148                 BL MOTEUR_DROIT_ON
149                 BL MOTEUR_GAUCHE_ON
150                 BL MOTEUR_DROIT_AVANT
151                 BL MOTEUR_GAUCHE_ARRIERE
152
153                 BL WAIT_1S
```

```
154                 BL WAIT_1S
155                 BL WAIT_1S
156                 BL WAIT_1S
157                 BL WAIT_0_1S
158                 BL WAIT_0_1S
159                 BL WAIT_0_1S
160                 BL WAIT_0_1S
161                 BL WAIT_0_1S
162                 BL WAIT_0_1S
163                 BL WAIT_0_1S
164                 BL WAIT_0_1S
165                 BL MOTEUR_DROIT_OFF
166                 BL MOTEUR_GAUCHE_OFF
167
168                 POP {LR}
169
170                 BX LR
171
172 DEP_HUITIEME_TOUR_R
173
174                 PUSH {LR}
175
176                 BL MOTEUR_DROIT_ON
177                 BL MOTEUR_GAUCHE_ON
178                 BL MOTEUR_DROIT_ARRIERE
179                 BL MOTEUR_GAUCHE_AVANT
180
181                 BL WAIT_1S
182                 BL WAIT_0_1S
183                 BL WAIT_0_1S
184                 BL WAIT_0_1S
185                 BL WAIT_0_1S
186
187                 BL MOTEUR_DROIT_OFF
188                 BL MOTEUR_GAUCHE_OFF
189
190                 POP {LR}
191
192                 BX LR
193
194 DEP_HUITIEME_TOUR_L
195
196                 PUSH {LR}
197
198                 BL MOTEUR_DROIT_ON
199                 BL MOTEUR_GAUCHE_ON
200                 BL MOTEUR_DROIT_AVANT
201                 BL MOTEUR_GAUCHE_ARRIERE
202
203                 BL WAIT_1S
204                 BL WAIT_0_1S
205                 BL WAIT_0_1S
206                 BL WAIT_0_1S
207                 BL WAIT_0_1S
208
209                 BL MOTEUR_DROIT_OFF
210                 BL MOTEUR_GAUCHE_OFF
211
```

```
212                 POP {LR}
213
214                 BX LR
215
216 DEP_SEIZIEME_TOUR_R
217
218                 PUSH {LR}
219
220                 BL MOTEUR_DROIT_ON
221                 BL MOTEUR_GAUCHE_ON
222                 BL MOTEUR_DROIT_ARRIERE
223                 BL MOTEUR_GAUCHE_AVANT
224
225                 BL WAIT_0_1S
226                 BL WAIT_0_1S
227                 BL WAIT_0_1S
228                 BL WAIT_0_1S
229                 BL WAIT_0_1S
230                 BL WAIT_0_1S
231                 BL WAIT_0_1S
232
233                 BL MOTEUR_DROIT_OFF
234                 BL MOTEUR_GAUCHE_OFF
235
236                 POP {LR}
237
238                 BX LR
239
240 DEP_SEIZIEME_TOUR_L
241
242                 PUSH {LR}
243
244                 BL MOTEUR_DROIT_ON
245                 BL MOTEUR_GAUCHE_ON
246                 BL MOTEUR_DROIT_AVANT
247                 BL MOTEUR_GAUCHE_ARRIERE
248
249                 BL WAIT_0_1S
250                 BL WAIT_0_1S
251                 BL WAIT_0_1S
252                 BL WAIT_0_1S
253                 BL WAIT_0_1S
254                 BL WAIT_0_1S
255                 BL WAIT_0_1S
256
257                 BL MOTEUR_DROIT_OFF
258                 BL MOTEUR_GAUCHE_OFF
259
260                 POP {LR}
261
262                 BX LR
263
264 DEP_LIGNE_DROITE
265
266                 PUSH {LR}
267
268                 BL MOTEUR_DROIT_ON
269                 BL MOTEUR_GAUCHE_ON
```

```
                BL MOTEUR_DROIT_AVANT
                BL MOTEUR_GAUCHE_AVANT

                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S

                BL MOTEUR_DROIT_OFF
                BL MOTEUR_GAUCHE_OFF

                POP {LR}

                BX LR

DEP_DEMI_LIGNE_DROITE

                PUSH {LR}

                BL MOTEUR_DROIT_ON
                BL MOTEUR_GAUCHE_ON
                BL MOTEUR_DROIT_AVANT
                BL MOTEUR_GAUCHE_AVANT

                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S


                BL MOTEUR_DROIT_OFF
                BL MOTEUR_GAUCHE_OFF

                POP {LR}

                BX LR


DEP_DEMI_CERCLE_R

                PUSH {LR}

                MOV r4, #16

                BL MOTEUR_DROIT_ON
                BL MOTEUR_GAUCHE_ON
                BL MOTEUR_DROIT_AVANT
                BL MOTEUR_GAUCHE_AVANT

loop1_dep_demi_cercle_r

                BL MOTEUR_DROIT_OFF
                BL WAIT_0_1S
```

```
328                 BL WAIT_0_1S
329                 BL WAIT_0_1S
330                 BL WAIT_0_1S
331                 BL WAIT_0_1S
332                 BL WAIT_0_1S
333 
334 
335                 BL MOTEUR_DROIT_ON
336                 BL WAIT_0_1S
337                 BL WAIT_0_1S
338 
339                 SUBS r4, #1
340                 CMP r4, #0
341                 BNE loop1_dep_demi_cercle_r
342 
343                 BL MOTEUR_DROIT_OFF
344                 BL MOTEUR_GAUCHE_OFF
345 
346                 POP {LR}
347 
348                 BX LR
349 
350 DEP_DEMI_CERCLE_L
351 
352                 PUSH {LR}
353 
354                 MOV r4, #16
355 
356                 BL MOTEUR_DROIT_ON
357                 BL MOTEUR_GAUCHE_ON
358                 BL MOTEUR_DROIT_AVANT
359                 BL MOTEUR_GAUCHE_AVANT
360 
361 loop1_dep_demi_cercle_l
362 
363                 BL MOTEUR_GAUCHE_OFF
364                 BL WAIT_0_1S
365                 BL WAIT_0_1S
366                 BL WAIT_0_1S
367                 BL WAIT_0_1S
368                 BL WAIT_0_1S
369                 BL WAIT_0_1S
370 
371 
372                 BL MOTEUR_GAUCHE_ON
373                 BL WAIT_0_1S
374                 BL WAIT_0_1S
375 
376                 SUBS r4, #1
377                 CMP r4, #0
378                 BNE loop1_dep_demi_cercle_l
379 
380                 BL MOTEUR_DROIT_OFF
381                 BL MOTEUR_GAUCHE_OFF
382 
383                 POP {LR}
384 
385                 BX LR
```

```
386
387 DEP_GRAND_DEMI_CERCLE_R
388
389                 PUSH {LR}
390
391                 MOV r4, #23
392
393                 BL MOTEUR_DROIT_ON
394                 BL MOTEUR_GAUCHE_ON
395                 BL MOTEUR_DROIT_AVANT
396                 BL MOTEUR_GAUCHE_AVANT
397
398 loop1_dep_grand_demi_cercle_r
399
400                 BL MOTEUR_DROIT_OFF
401                 BL WAIT_0_1S
402                 BL WAIT_0_1S
403                 BL WAIT_0_1S
404                 BL WAIT_0_1S
405
406
407                 BL MOTEUR_DROIT_ON
408                 BL WAIT_0_1S
409                 BL WAIT_0_1S
410                 BL WAIT_0_1S
411                 BL WAIT_0_1S
412
413                 SUBS r4, #1
414                 CMP r4, #0
415                 BNE loop1_dep_grand_demi_cercle_r
416
417                 BL MOTEUR_DROIT_OFF
418                 BL MOTEUR_GAUCHE_OFF
419
420
421                 POP {LR}
422
423                 BX LR
424
425 DEP_GRAND_DEMI_CERCLE_L
426
427                 PUSH {LR}
428
429                 MOV r4, #23
430
431                 BL MOTEUR_DROIT_ON
432                 BL MOTEUR_GAUCHE_ON
433                 BL MOTEUR_DROIT_AVANT
434                 BL MOTEUR_GAUCHE_AVANT
435
436 loop1_dep_grand_demi_cercle_l
437
438                 BL MOTEUR_GAUCHE_OFF
439                 BL WAIT_0_1S
440                 BL WAIT_0_1S
441                 BL WAIT_0_1S
442                 BL WAIT_0_1S
443
```

```
444
445                 BL MOTEUR_GAUCHE_ON
446                 BL WAIT_0_1S
447                 BL WAIT_0_1S
448                 BL WAIT_0_1S
449                 BL WAIT_0_1S
450
451                 SUBS r4, #1
452                 CMP r4, #0
453                 BNE loop1_dep_grand_demi_cercle_l
454
455                 BL MOTEUR_DROIT_OFF
456                 BL MOTEUR_GAUCHE_OFF
457
458
459                 POP {LR}
460
461                 BX LR
462
463 ; -------------------------------------
464 ; ---- DEBUT DEPLACEMENTS LETTRES ----
465 ; -------------------------------------
466
467 DEP_LETTRE_A
468
469                 PUSH {LR}
470
471                 BL DEP_SEIZIEME_TOUR_L
472
473                 ; On avance en ligne droite
474                 BL DEP_LIGNE_DROITE
475
476                 BL DEP_QUART_TOUR_L
477                 BL DEP_HUITIEME_TOUR_L
478
479                 ; On avance en ligne droite
480                 BL DEP_LIGNE_DROITE
481
482                 BL DEP_DEMI_TOUR_R
483
484                 BL MOTEUR_DROIT_ON
485                 BL MOTEUR_GAUCHE_ON
486                 BL MOTEUR_DROIT_AVANT
487                 BL MOTEUR_GAUCHE_AVANT
488
489                 BL WAIT_1S
490                 BL WAIT_1S
491                 BL WAIT_1S
492                 BL WAIT_1S
493                 BL WAIT_1S
494
495                 BL DEP_SEIZIEME_TOUR_R
496                 BL DEP_SEIZIEME_TOUR_R
497                 BL DEP_SEIZIEME_TOUR_R
498
499                 BL MOTEUR_DROIT_ON
500                 BL MOTEUR_GAUCHE_ON
501                 BL MOTEUR_DROIT_AVANT
```

```
                BL MOTEUR_GAUCHE_AVANT

                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S

                BL DEP_SEIZIEME_TOUR_R
                BL DEP_SEIZIEME_TOUR_R

                BL MOTEUR_DROIT_ON
                BL MOTEUR_GAUCHE_ON
                BL MOTEUR_DROIT_AVANT
                BL MOTEUR_GAUCHE_AVANT

                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S
                BL WAIT_1S

                BL MOTEUR_DROIT_OFF
                BL MOTEUR_GAUCHE_OFF


                POP {LR}

                BX LR

DEP_LETTRE_B

                PUSH {LR}

                BL DEP_LIGNE_DROITE
                BL DEP_QUART_TOUR_R
                BL DEP_DEMI_CERCLE_R
                BL DEP_DEMI_TOUR_L
                BL DEP_DEMI_CERCLE_R

                POP {LR}

                BX LR

DEP_LETTRE_C

                PUSH {LR}

                BL DEP_QUART_TOUR_L
                BL DEP_GRAND_DEMI_CERCLE_R


                POP {LR}

                BX LR

DEP_LETTRE_D

                PUSH {LR}
```

```
560
561                 BL DEP_LIGNE_DROITE
562                 BL DEP_QUART_TOUR_R
563                 BL DEP_GRAND_DEMI_CERCLE_R
564
565                 POP {LR}
566
567                 BX LR
568
569 DEP_LETTRE_E
570
571                 PUSH {LR}
572
573                 MOV r4, #2
574 loop1_dep_lettre_e
575
576                 BL DEP_QUART_TOUR_R
577                 BL DEP_DEMI_LIGNE_DROITE
578                 BL DEP_DEMI_TOUR_L
579                 BL DEP_DEMI_LIGNE_DROITE
580                 BL DEP_QUART_TOUR_R
581                 BL DEP_DEMI_LIGNE_DROITE
582
583                 SUBS r4, #1
584                 CMP r4, #0
585                 BNE loop1_dep_lettre_e
586
587                 BL DEP_QUART_TOUR_R
588                 BL DEP_DEMI_LIGNE_DROITE
589
590                 POP {LR}
591
592                 BX LR
593
594 DEP_LETTRE_F
595
596                 PUSH {LR}
597
598                 MOV r4, #2
599 loop1_dep_lettre_f
600
601                 BL DEP_DEMI_LIGNE_DROITE
602                 BL DEP_QUART_TOUR_R
603                 BL DEP_DEMI_LIGNE_DROITE
604                 BL DEP_DEMI_TOUR_L
605                 BL DEP_DEMI_LIGNE_DROITE
606                 BL DEP_QUART_TOUR_R
607
608                 SUBS r4, #1
609                 CMP r4, #0
610                 BNE loop1_dep_lettre_f
611
612
613                 POP {LR}
614
615                 BX LR
616
617 DEP_LETTRE_G
```

```
618
619                 PUSH {LR}
620
621                 BL DEP_QUART_TOUR_L
622                 BL DEP_GRAND_DEMI_CERCLE_R
623                 BL DEP_DEMI_TOUR_L
624                 BL DEP_GRAND_DEMI_CERCLE_L
625
626                 BL DEP_HUITIEME_TOUR_L
627                 BL DEP_DEMI_LIGNE_DROITE
628
629                 BL DEP_SEIZIEME_TOUR_R
630                 BL DEP_DEMI_LIGNE_DROITE
631                 BL DEP_DEMI_TOUR_L
632                 BL DEP_DEMI_LIGNE_DROITE
633                 BL DEP_DEMI_LIGNE_DROITE
634
635                 POP {LR}
636
637                 BX LR
638
639 DEP_LETTRE_H
640
641                 PUSH {LR}
642
643                 BL DEP_LIGNE_DROITE
644                 BL DEP_DEMI_TOUR_L
645                 BL DEP_DEMI_LIGNE_DROITE
646                 BL DEP_QUART_TOUR_L
647
648                 BL DEP_DEMI_LIGNE_DROITE
649
650                 BL DEP_QUART_TOUR_L
651                 BL DEP_DEMI_LIGNE_DROITE
652                 BL DEP_DEMI_TOUR_L
653                 BL DEP_LIGNE_DROITE
654
655                 POP {LR}
656
657                 BX LR
658
659 DEP_LETTRE_I
660
661                 PUSH {LR}
662
663                 BL DEP_LIGNE_DROITE
664
665                 POP {LR}
666
667                 BX LR
668
669 DEP_LETTRE_J
670
671                 PUSH {LR}
672
673                 BL DEP_DEMI_TOUR_R
674                 BL DEP_DEMI_CERCLE_L
675                 BL DEP_LIGNE_DROITE
```

```
676
677                 POP {LR}
678
679                 BX LR
680
681 DEP_LETTRE_K
682
683                 PUSH {LR}
684
685                 BL DEP_DEMI_LIGNE_DROITE
686
687                 BL DEP_QUART_TOUR_R
688                 BL DEP_HUITIEME_TOUR_R
689                 BL DEP_DEMI_LIGNE_DROITE
690                 BL DEP_DEMI_TOUR_L
691                 BL DEP_DEMI_LIGNE_DROITE
692
693                 BL DEP_QUART_TOUR_R
694                 BL DEP_DEMI_LIGNE_DROITE
695                 BL DEP_DEMI_TOUR_L
696                 BL DEP_DEMI_LIGNE_DROITE
697
698                 BL DEP_HUITIEME_TOUR_R
699                 BL DEP_QUART_TOUR_R
700                 BL DEP_SEIZIEME_TOUR_L
701                 BL DEP_DEMI_LIGNE_DROITE
702
703                 POP {LR}
704
705                 BX LR
706
707 DEP_LETTRE_L
708
709                 PUSH {LR}
710
711                 BL DEP_QUART_TOUR_L
712                 BL DEP_DEMI_LIGNE_DROITE
713                 BL DEP_QUART_TOUR_R
714                 BL DEP_LIGNE_DROITE
715
716                 POP {LR}
717
718                 BX LR
719
720 DEP_LETTRE_M
721
722                 PUSH {LR}
723
724                 BL DEP_LIGNE_DROITE
725
726                 BL DEP_QUART_TOUR_R
727                 BL DEP_HUITIEME_TOUR_R
728                 BL DEP_DEMI_LIGNE_DROITE
729
730                 BL DEP_QUART_TOUR_L
731                 BL DEP_SEIZIEME_TOUR_L
732                 BL DEP_DEMI_LIGNE_DROITE
733
```

```
734                 BL DEP_QUART_TOUR_R
735                 BL DEP_HUITIEME_TOUR_R
736                 BL DEP_LIGNE_DROITE
737 
738                 POP {LR}
739 
740                 BX LR
741 
742 DEP_LETTRE_N
743 
744                 PUSH {LR}
745 
746                 BL DEP_LIGNE_DROITE
747 
748                 BL DEP_QUART_TOUR_R
749                 BL DEP_HUITIEME_TOUR_R
750                 BL DEP_LIGNE_DROITE
751                 BL DEP_DEMI_LIGNE_DROITE
752 
753                 BL DEP_QUART_TOUR_L
754                 BL DEP_HUITIEME_TOUR_L
755                 BL DEP_LIGNE_DROITE
756 
757                 POP {LR}
758 
759                 BX LR
760 
761 DEP_LETTRE_O
762 
763                 PUSH {LR}
764 
765                 BL DEP_QUART_TOUR_L
766                 BL DEP_GRAND_DEMI_CERCLE_R
767                 BL DEP_GRAND_DEMI_CERCLE_R
768 
769                 POP {LR}
770 
771                 BX LR
772 
773 DEP_LETTRE_P
774 
775                 PUSH {LR}
776 
777                 BL DEP_LIGNE_DROITE
778                 BL DEP_QUART_TOUR_R
779                 BL DEP_DEMI_CERCLE_R
780 
781                 POP {LR}
782 
783                 BX LR
784 
785 DEP_LETTRE_Q
786 
787                 PUSH {LR}
788 
789                 BL DEP_LETTRE_O
790                 BL DEP_QUART_TOUR_L
791                 BL DEP_HUITIEME_TOUR_L
```

```
792                 BL DEP_DEMI_LIGNE_DROITE
793
794                 POP {LR}
795
796                 BX LR
797
798 DEP_LETTRE_R
799
800                 PUSH {LR}
801
802                 BL DEP_LETTRE_P
803                 BL DEP_QUART_TOUR_L
804                 BL DEP_HUITIEME_TOUR_L
805                 BL DEP_SEIZIEME_TOUR_L
806                 BL DEP_DEMI_LIGNE_DROITE
807
808                 POP {LR}
809
810                 BX LR
811
812 DEP_LETTRE_S
813
814                 PUSH {LR}
815
816                 BL DEP_DEMI_TOUR_R
817                 BL DEP_DEMI_CERCLE_L
818                 BL DEP_HUITIEME_TOUR_L
819                 BL DEP_LIGNE_DROITE
820                 BL DEP_HUITIEME_TOUR_R
821                 BL DEP_DEMI_CERCLE_R
822
823                 POP {LR}
824
825                 BX LR
826
827 DEP_LETTRE_T
828
829                 PUSH {LR}
830
831                 BL DEP_LIGNE_DROITE
832                 BL DEP_QUART_TOUR_R
833                 BL DEP_DEMI_LIGNE_DROITE
834                 BL DEP_DEMI_TOUR_L
835                 BL DEP_DEMI_LIGNE_DROITE
836                 BL DEP_DEMI_LIGNE_DROITE
837
838                 POP {LR}
839
840                 BX LR
841
842 DEP_LETTRE_U
843
844                 PUSH {LR}
845
846                 BL DEP_LIGNE_DROITE
847                 BL DEP_DEMI_TOUR_R
848                 BL DEP_LIGNE_DROITE
849                 BL DEP_DEMI_CERCLE_L
```

```
850                 BL DEP_LIGNE_DROITE
851
852                 POP {LR}
853
854                 BX LR
855
856 DEP_LETTRE_V
857
858                 PUSH {LR}
859
860                 BL DEP_SEIZIEME_TOUR_L
861                 BL DEP_LIGNE_DROITE
862                 BL DEP_DEMI_TOUR_R
863                 BL DEP_LIGNE_DROITE
864
865                 BL DEP_QUART_TOUR_L
866                 BL DEP_HUITIEME_TOUR_L
867                 BL DEP_LIGNE_DROITE
868
869                 POP {LR}
870
871                 BX LR
872
873 DEP_LETTRE_W
874
875                 PUSH {LR}
876
877                 BL DEP_SEIZIEME_TOUR_R
878                 BL DEP_LIGNE_DROITE
879                 BL DEP_DEMI_TOUR_L
880                 BL DEP_LIGNE_DROITE
881
882                 BL DEP_QUART_TOUR_R
883                 BL DEP_DEMI_LIGNE_DROITE
884
885                 BL DEP_QUART_TOUR_L
886                 BL DEP_DEMI_LIGNE_DROITE
887                 BL DEP_QUART_TOUR_R
888                 BL DEP_SEIZIEME_TOUR_R
889                 BL DEP_LIGNE_DROITE
890
891                 POP {LR}
892
893                 BX LR
894
895 DEP_LETTRE_X
896
897                 PUSH {LR}
898
899                 BL DEP_HUITIEME_TOUR_L
900
901                 BL DEP_DEMI_LIGNE_DROITE
902                 BL DEP_DEMI_LIGNE_DROITE
903
904                 BL DEP_DEMI_TOUR_L
905
906                 BL DEP_DEMI_LIGNE_DROITE
907
```

```
908                 BL DEP_QUART_TOUR_R
909
910                 BL DEP_DEMI_LIGNE_DROITE
911
912                 BL DEP_DEMI_TOUR_L
913
914                 BL DEP_DEMI_LIGNE_DROITE
915                 BL DEP_DEMI_LIGNE_DROITE
916
917                 POP {LR}
918
919                 BX LR
920
921 DEP_LETTRE_Y
922
923                 PUSH {LR}
924
925                 BL DEP_DEMI_LIGNE_DROITE
926
927                 BL DEP_SEIZIEME_TOUR_R
928                 BL DEP_DEMI_LIGNE_DROITE
929
930                 BL DEP_DEMI_TOUR_L
931                 BL DEP_DEMI_LIGNE_DROITE
932
933                 BL DEP_QUART_TOUR_R
934                 BL DEP_SEIZIEME_TOUR_R
935                 BL DEP_DEMI_LIGNE_DROITE
936
937                 POP {LR}
938
939                 BX LR
940
941 DEP_LETTRE_Z
942
943                 PUSH {LR}
944
945                 BL DEP_QUART_TOUR_L
946                 BL DEP_LIGNE_DROITE
947
948                 BL DEP_QUART_TOUR_R
949                 BL DEP_HUITIEME_TOUR_R
950
951                 BL DEP_LIGNE_DROITE
952                 BL DEP_DEMI_LIGNE_DROITE
953
954                 BL DEP_QUART_TOUR_L
955                 BL DEP_HUITIEME_TOUR_L
956
957                 BL DEP_LIGNE_DROITE
958
959                 POP {LR}
960
961                 BX LR
962
963                 END
```